

CS-104 Fundamentals of Programming

Instructor: Asst

Professor Jahan Zeb
Department of Computer & Software Engineering (DC&SE)
College of E&ME
NUST

Control Structures

- Sequential execution
 - Statements executed in order
- Transfer of control
 - Next statement executed *not* next one in sequence
- 3 control structures
 - Sequence structure
 - Programs executed sequentially by default
 - Selection structures
 - **if, if/else, switch**
 - Repetition structures
 - **while, do/while, for**

Control Structures

□ Flowchart

- Graphical representation of an algorithm
- Special-purpose symbols connected by arrows (flowlines)
- Rectangle symbol (action symbol)
 - Any type of action
- Oval symbol
 - Beginning or end of a program

if Selection Structure

□ Selection structure

- Choose among alternative courses of action
- Pseudocode example:

If student's grade is greater than or equal to 60
Print "Passed"

- If the condition is **true**
 - Print statement executed, program continues to next statement
- If the condition is **false**
 - Print statement ignored, program continues
- Indenting makes programs easier to read
 - C++ ignores whitespace characters (tabs, spaces, etc.)

if Selection Structure

□ Translation into C++

*If student's grade is greater than or equal to 60
Print "Passed"*

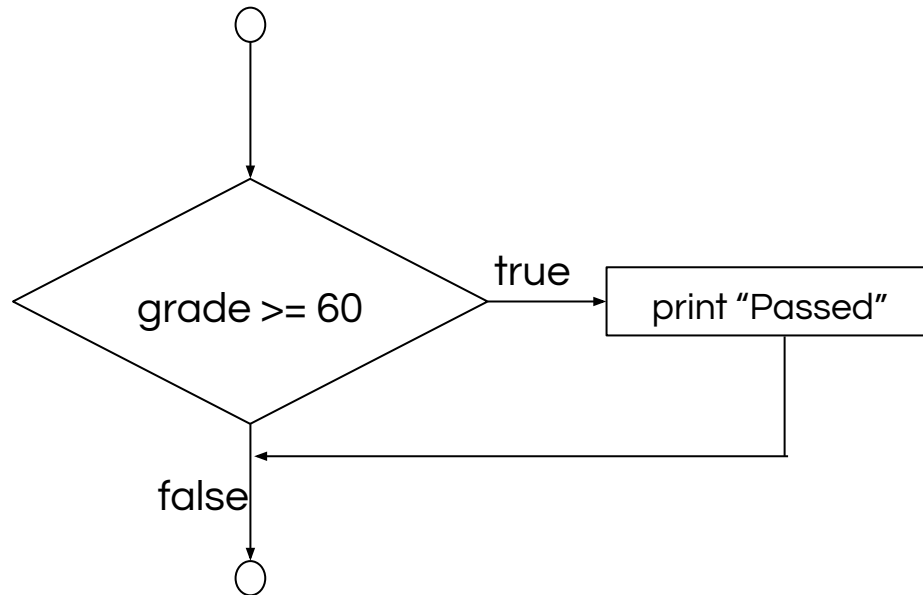
```
if ( grade >= 60 )  
    cout << "Passed";
```

□ Diamond symbol (decision symbol)

- Indicates decision is to be made
- Contains an expression that can be true or false
 - Test condition, follow path

if Selection Structure

□ Flowchart of pseudocode statement



if/else Selection Structure

□ **if**

- Performs action if condition true

□ **if/else**

- Different actions if conditions true or false

□ Pseudocode

if student's grade is greater than or equal to 60
print "Passed"

else

print "Failed"

□ C++ code

```
if ( grade >= 60 )  
    cout << "Passed";  
else  
    cout << "Failed";
```

Selection Structure

□ Ternary conditional operator (?:)

- Three arguments (condition, value if **true**, value if **false**)

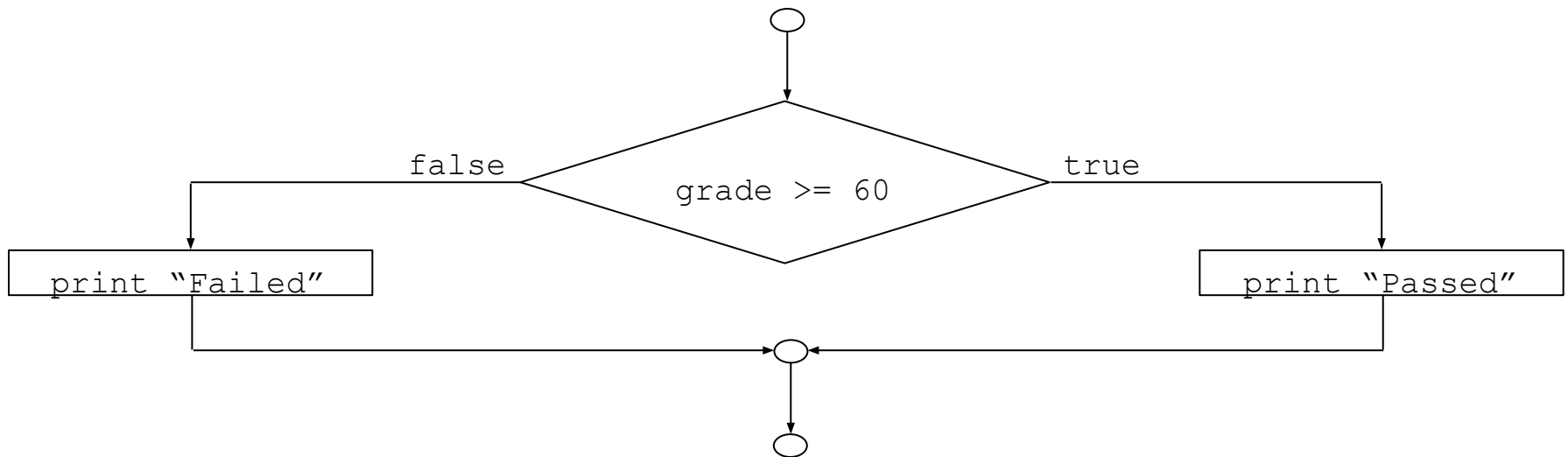
□ Code could be written:

```
cout << ( grade >= 60 ? "Passed" : "Failed" );
```

↑
Condition

↑
Value if true

↑
Value if false



if/else Selection Structure

□ Nested **if/else** structures

- One inside another, test for multiple cases
- Once condition met, other statements skipped

if student's grade is greater than or equal to 90

Print "A"

else

if student's grade is greater than or equal to 80

Print "B"

else

if student's grade is greater than or equal to 70

Print "C"

else

if student's grade is greater than or equal to 60

Print "D"

else

Print "F"

if/else Selection Structure

□ Example

```
if ( grade >= 90 )           // 90 and above
    cout << "A";
else if ( grade >= 80 )      // 80-89
    cout << "B";
else if ( grade >= 70 )      // 70-79
    cout << "C";
else if ( grade >= 60 )      // 60-69
    cout << "D";
else                          // less than 60
    cout << "F";
```

if/else Selection Structure

□ Compound statement

- Set of statements within a pair of braces

```
if ( grade >= 60 )  
    cout << "Passed.\n";  
else {  
    cout << "Failed.\n";  
    cout << "You must take this course again.\n";  
}
```

- Without braces,

```
cout << "You must take this course again.\n";  
always executed
```

□ Block

- Set of statements within braces

Even or Odd

```
int num;  
cout<<"Enter any number"<<endl;  
cin>>num;  
  
if(num%2==0)  
    cout<<"The number is even"<<endl;  
else  
    cout<<"The number is odd"<<endl;
```

Dangling Else Problem

```
if ( x > 5)
    if ( y > 5)
        cout << "x and y are > 5";
else
    cout << "x is <= 5";
```

Dangling Else Problem

```
if ( x > 5)
    if ( y > 5)
        cout << "x and y are > 5";
    else
        cout << "x is <= 5";
```

Dangling Else Problem

```
if ( x > 5)  
    {  
        if ( y > 5)  
            cout << “x and y are > 5”;  
        }  
else  
    cout << “x is <= 5”;
```

□ **else** associated with the first **if**

while Repetition Structure

□ Repetition structure

- Action repeated while some condition remains true
- Psuedocode

while there are more items on my shopping list

Purchase next item and cross it off my list

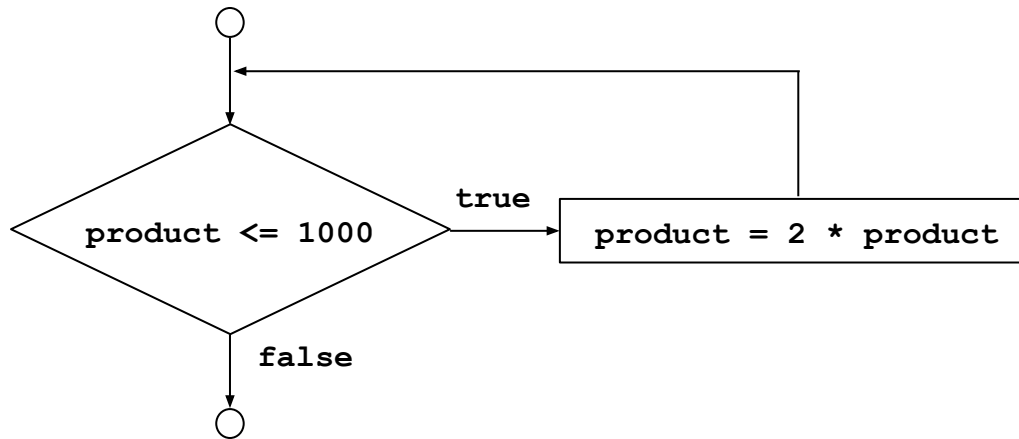
- **while** loop repeated until condition becomes false

□ Example

```
int product = 2;  
while ( product <= 1000 )  
    product = 2 * product;
```

The while Repetition Structure

□ Flowchart of **while** loop



Formulating Algorithms (Counter-Controlled Repetition)

□ Counter-controlled repetition

- Loop repeated until counter reaches certain value

□ Definite repetition

- Number of repetitions known

□ Example

A class of ten students took a quiz. The grades (integers in the range 0 to 100) for this quiz are available to you. Determine the class average on the quiz.

Formulating Algorithms (Counter-Controlled Repetition)

□ Pseudocode for example:

Set total to zero

Set grade counter to one

While grade counter is less than or equal to ten

Input the next grade

Add the grade into the total

Add one to the grade counter

Set the class average to the total divided by ten

Print the class average

□ Next: C++ code for this example

```
1
2 // Class average program with counter-controlled repetition.
3 #include <iostream>
4
5 using std::cout;
6 using std::cin;
7 using std::endl;
8
9 // function main begins program execution
10 int main()
11 {
12     int total;           // sum of grades input by user
13     int gradeCounter;    // number of grade to be entered next
14     int grade;           // grade value
15     int average;         // average of grades
16
17     // initialization phase
18     total = 0;           // initialize total
19     gradeCounter = 1;    // initialize loop counter
20
```

```

21 // processing phase
22 while ( gradeCounter <= 10 ) {           // loop 10 times
23     cout << "Enter grade: ";           // prompt for input
24     cin >> grade;                       // read grade from user
25     total = total + grade;              // add grade to total
26     gradeCounter = gradeCounter + 1;    // increment counter
27 }
28
29 // termination phase
30 average = total / 10;                   // integer division
31
32 // display result
33 cout << "Class average is ";
34
35 return 0;    // indicate p
36
37 } // end function main

```

The counter gets incremented each time the loop executes. Eventually, the counter causes the loop to end.

```

Enter grade: 98
Enter grade: 76
Enter grade: 71
Enter grade: 87
Enter grade: 83
Enter grade: 90
Enter grade: 57
Enter grade: 79
Enter grade: 82
Enter grade: 94
Class average is 81

```

Formulating Algorithms (Sentinel-Controlled Repetition)

□ Suppose problem becomes:

Develop a class-averaging program that will process an arbitrary number of grades each time the program is run

- Unknown number of students
- How will program know when to end?

□ Sentinel value

- Indicates “end of data entry”
- Loop ends when sentinel input
- Sentinel chosen so it cannot be confused with regular input
 - -1 in this case

Formulating Algorithms (Sentinel-Controlled Repetition)

- Top-down, stepwise refinement
 - Begin with pseudocode representation of top
 - Determine the class average for the quiz*
 - Divide top into smaller tasks, list in order
 - Initialize variables*
 - Input, sum and count the quiz grades*
 - Calculate and print the class average*

Formulating Algorithms (Sentinel-Controlled Repetition)

- Many programs have three phases
 - Initialization
 - Initializes the program variables
 - Processing
 - Input data, adjusts program variables
 - Termination
 - Calculate and print the final results
 - Helps break up programs for top-down refinement

Formulating Algorithms (Sentinel-Controlled Repetition)

□ Refine the initialization phase

Initialize variables

becomes:

Initialize total to zero

Initialize counter to zero

□ Processing

Input, sum and count the quiz grades

becomes:

Input the first grade (possibly the sentinel)

While the user has not as yet entered the sentinel

Add this grade into the running total

Add one to the grade counter

Input the next grade (possibly the sentinel)

Formulating Algorithms (Sentinel-Controlled Repetition)

□ Termination

Calculate and print the class average

becomes:

If the counter is not equal to zero

Set the average to the total divided by the counter

Print the average

Else

Print “No grades were entered”

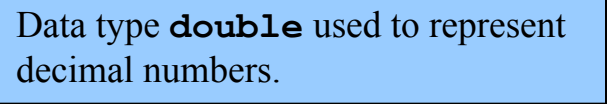
□ Next: C++ program

```

1 // Fig. 2.9: fig02_09.cpp
2 // Class average program with sentinel-controlled repetition.
3 #include <iostream>
4
5 using std::cout;
6 using std::cin;
7 using std::endl;
8
9
10 #include <iomanip>           // parameterized stream manipulators
11
12 using std::setprecision;   // sets numeric output precision
13
14 // function main begins program execution
15 int main()
16 {
17     int total;              // sum of grades
18     int gradeCounter;      // number of grades entered
19     int grade;              // grade value
20
21     double average;        // number with decimal point for average
22
23     // initialization phase
24     total = 0;              // initialize total
25     gradeCounter = 0;      // initialize loop counter

```

Data type **double** used to represent decimal numbers.



```
26
27 // processing phase
28 // get first grade from user
29 cout << "Enter grade, -1 to end: "; // prompt for input
30 cin >> grade; // read grade from user
```

```
31
32 // loop until sentinel value read from user
```

```
33 while ( grade != -1 )
34     total = total + grade;
35     gradeCounter = gradeCounter + 1;
36
37     cout << "Enter grade, -1 to end: ";
38     cin >> grade;
39
40 } // end while
```

```
41
42 // termination phase
```

```
43 // if user entered at least one grade ...
```

```
44 if ( gradeCounter != 0 ) {
```

```
45
46     // calculate average of all grades entered
```

```
47     average = static_cast< double >( total ) / gradeCounter;
48
```

`static_cast<double>()` treats `total` as a `double` temporarily (casting).

Required because dividing two integers truncates the remainder.

`gradeCounter` is an `int`, but it gets *promoted* to `double`.

```
49 // display average with two digits of precision
50 cout << "Class average is " << setprecision( 2 )
51     << average << endl;
52
53 } // end if part of if/else
54
55 else // if no grades were entered, output appropriate message
56     cout << "No grades were entered" << endl;
57
58 return 0; // indicate program ended successfully
59
60 } // end function main
```

```
Enter grade, -1 to end: 75
Enter grade, -1 to end: 94
Enter grade, -1 to end: 97
Enter grade, -1 to end: 88
Enter grade, -1 to end: 70
Enter grade, -1 to end: 64
Enter grade, -1 to end: 83
Enter grade, -1 to end: 89
Enter grade, -1 to end: -1
Class average is 82.50
```

setprecision(2) prints two digits past decimal point (rounded to fit precision).

Programs that use this must include **<iomanip>**

Nested Control Structures

□ Problem statement

A college has a list of test results (1 = pass, 2 = fail) for 10 students. Write a program that analyzes the results. If more than 8 students pass, print “Hardworking Class”.

□ Notice that

- Program processes 10 results
 - Fixed number, use counter-controlled loop
- Two counters can be used
 - One counts number that passed
 - Another counts number that fail
- Each test result is 1 or 2
 - If not 1, assume 2

Nested Control Structures

□ Top level outline

Analyze exam results and decide if class was hardworking

□ First refinement

Initialize variables

Input the ten quiz grades and count passes and failures

Print a summary of the exam results and decide if class was hardworking

□ Refine

Initialize variables

becomes:

Initialize passes to zero

Initialize failures to zero

Initialize student counter to one

Nested Control Structures

□ Refine

Input the ten quiz grades and count passes and failures

becomes:

While student counter is less than or equal to ten

Input the next exam result

If the student passed

Add one to passes

Else

Add one to failures

Add one to student counter

Nested Control Structures

□ Refine

*Print a summary of the exam results and decide if class was
hardworking*

becomes:

Print the number of passes

Print the number of failures

If more than eight students passed

Print “Hardworking Class”

□ Program next

```
1 // Fig. 2.11: fig02_11.cpp
2 // Analysis of examination results.
3 #include <iostream>
4
5 using std::cout;
6 using std::cin;
7 using std::endl;
8
9 // function main begins program execution
10 int main()
11 {
12     // initialize variables in declarations
13     int passes = 0;           // number of passes
14     int failures = 0;         // number of failures
15     int studentCounter = 1;   // student counter
16     int result;               // one exam result
17
18     // process 10 students using counter-controlled loop
19     while ( studentCounter <= 10 ) {
20
21         // prompt user for input and obtain value from user
22         cout << "Enter result (1 = pass, 2 = fail): ";
23         cin >> result;
24     }
```

```
25 // if result 1, increment passes; if/else nested in while
26 if ( result == 1 ) // if/else nested in while
27     passes = passes + 1;
28
29 else // if result not 1, increment failures
30     failures = failures + 1;
31
32 // increment studentCounter so loop eventually terminates
33 studentCounter = studentCounter + 1;
34
35 } // end while
36
37 // termination phase; display number of passes and failures
38 cout << "Passed " << passes << endl;
39 cout << "Failed " << failures << endl;
40
41 // if more than eight students passed, print "raise tuition"
42 if ( passes > 8 )
43     cout << "Hardworking Class" << endl;
44
45 return 0; // successful termination
46
47 } // end function main
```

Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Passed 6
Failed 4

Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Passed 9
Failed 1
Hardworking Class